01 Diagrama de casos de uso

> Recoge qué puede hacer cada actor con Recetario. Los casos de uso se agrupan por bloque funcional y cada uno enlaza con una historia de usuario del backlog (US-xx).

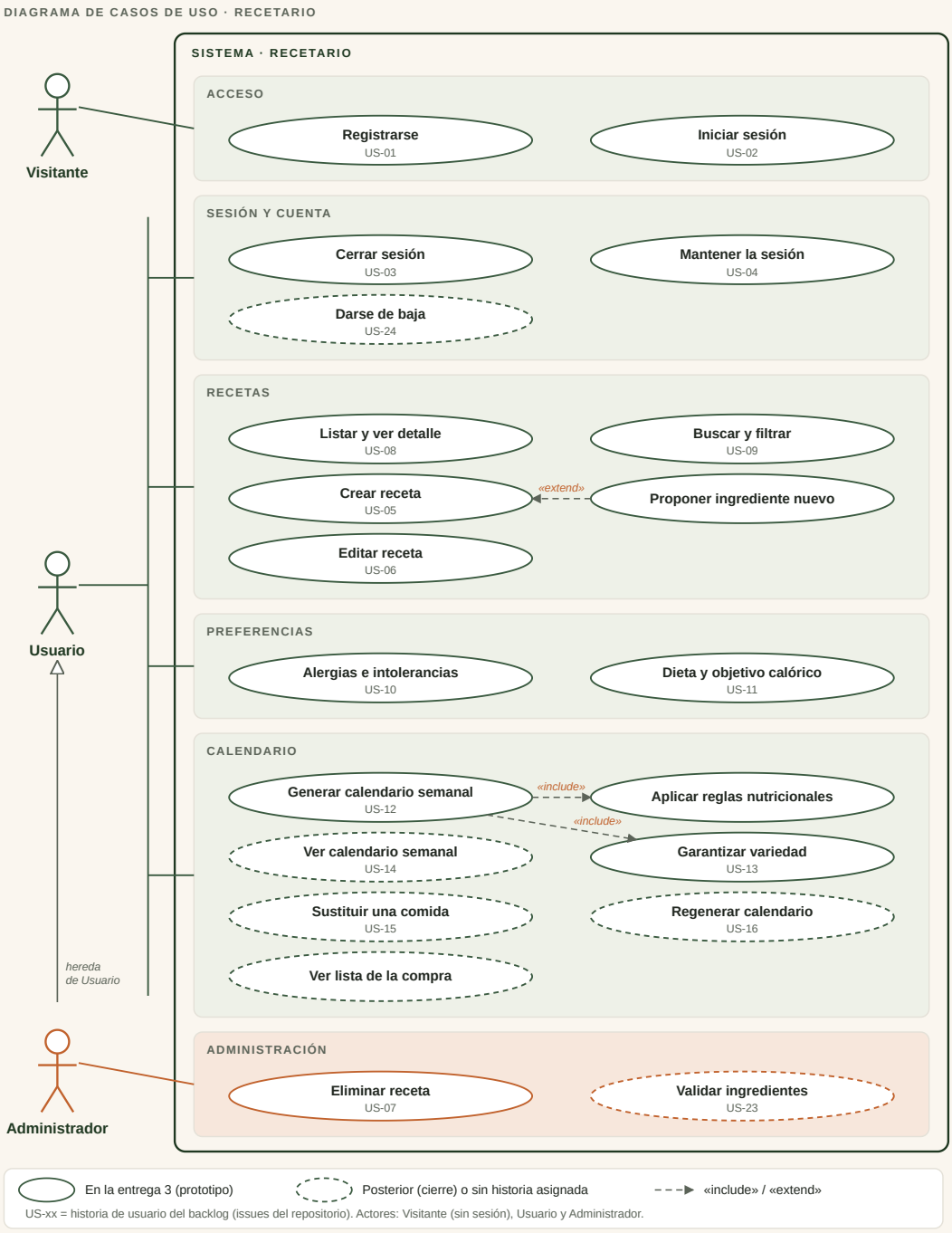


DIAGRAMA DE CASOS DE USO · 3 ACTORES

Visitante — sin sesión: solo puede registrarse o iniciar sesión.

Usuario — gestiona recetas, preferencias y su calendario semanal.

Administrador — hereda todo lo del usuario y además elimina recetas y valida ingredientes propuestos.

02 Esquema relacional

> Traducción del modelo entidad-relación a las 11 tablas de PostgreSQL que gestiona Prisma. Las tablas puente resuelven las relaciones N:M y las claves foráneas indican a qué tabla apuntan.

ESQUEMA RELACIONAL · POSTGRESQL (PRISMA)

IDENTIDAD Y PREFERENCIAS

usuario

PK	**id**	INT
	email	VARCHAR · UQ
	password_hash	VARCHAR
	nombre	VARCHAR
	rol	VARCHAR · «usuario»
	creado_en	TIMESTAMP

preferencia_nutricional

PK	**id**	INT
FK	usuario_id	INT · UQ → usuario.id
	tipo_dieta	VARCHAR
	objetivo_calorico_diario	INT

alergia

PK	**id**	INT
	nombre	VARCHAR · UQ

usuario_alergia

PF	**usuario_id**	INT → usuario.id
PF	**alergia_id**	INT → alergia.id

CATÁLOGO DE RECETAS

ingrediente

PK	**id**	INT
	nombre	VARCHAR · UQ
	calorias_100g	INT
	dieta_minima	VARCHAR · «Omnívora»
	validado	BOOLEAN · true
FK	propuesto_por_id	INT? → usuario.id

ingrediente_alergia

PF	**ingrediente_id**	INT → ingrediente.id
PF	**alergia_id**	INT → alergia.id

receta

PK	**id**	INT
FK	usuario_id	INT? → usuario.id
	nombre	VARCHAR
	tipo_comida	VARCHAR
	tiempo_preparacion_min	INT
	descripcion	VARCHAR

receta_ingrediente

PF	**receta_id**	INT → receta.id
PF	**ingrediente_id**	INT → ingrediente.id
	cantidad	DOUBLE
	unidad	VARCHAR

paso_receta

PK	**id**	INT
FK	receta_id	INT → receta.id
	orden	INT · UQ(receta, orden)
	texto	VARCHAR

PLANIFICACIÓN SEMANAL

calendario_semanal

PK	**id**	INT
FK	usuario_id	INT → usuario.id
	fecha_inicio	TIMESTAMP

entrada_calendario

PK	**id**	INT
FK	calendario_id	INT → calendario_semanal.id
	dia_semana	VARCHAR
	tipo_comida	VARCHAR
FK	receta_id	INT? → receta.id

UQ (calendario_id, dia_semana, tipo_comida)

LEYENDA Y BORRADO

PK Clave primaria
FK Clave foránea
PF PK compuesta formada por FK

tipo? = columna opcional (NULL)

Al borrar el registro padre

- CASCADE: tablas puente, pasos, ingredientes de receta, calendarios y preferencias.
- SET NULL: autor de receta y receta de entrada.

ESQUEMA RELACIONAL · 11 TABLAS

Lista de la compra — no es una tabla: se calcula sumando los ingredientes de las recetas del calendario.

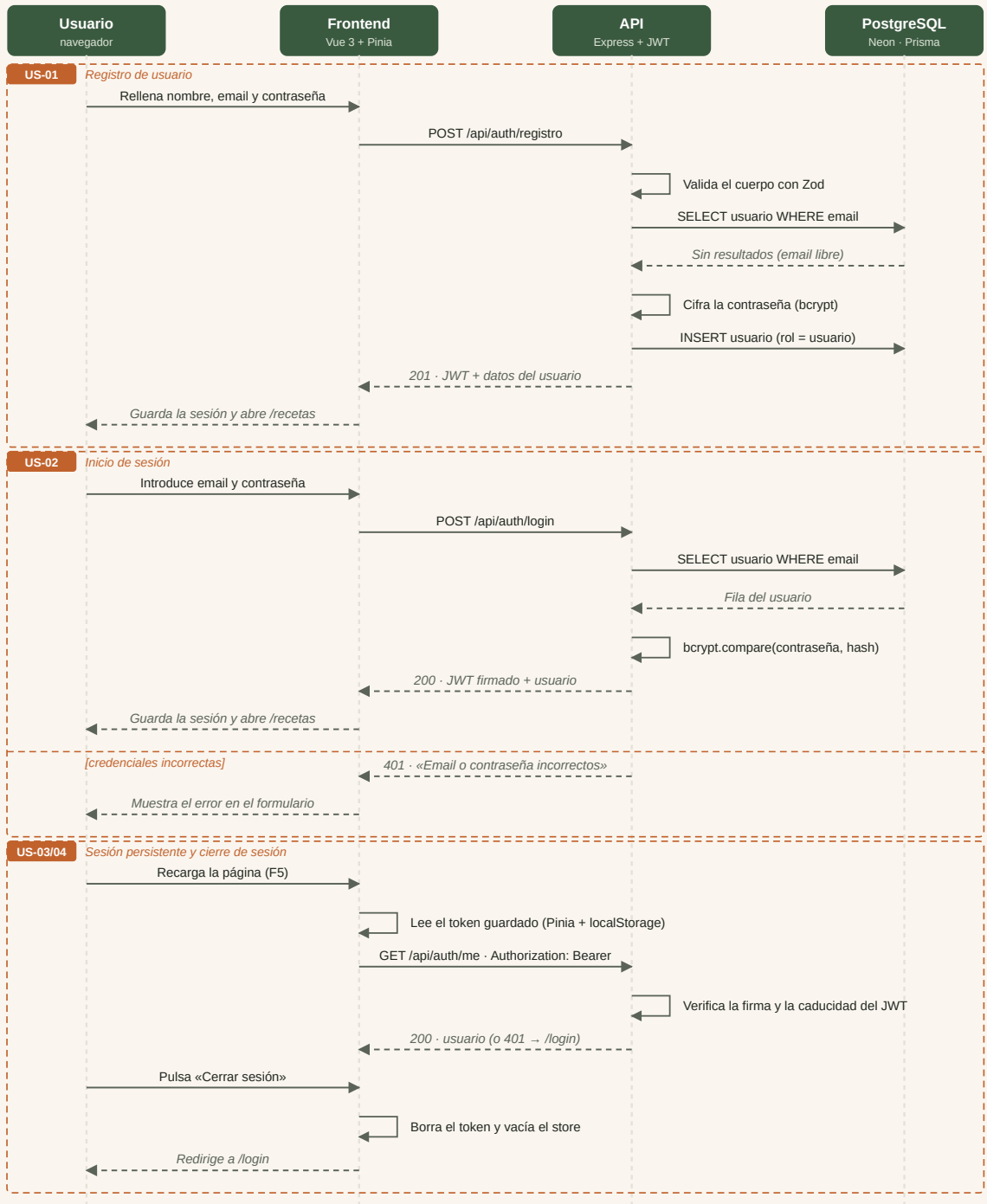
Recetas públicas — el autor es opcional (SET NULL) para conservar la receta si el usuario se da de baja.

Ingredientes — los propone un usuario y los valida el administrador (validado, propuesto_por_id).

03 Secuencia: autenticación

> Registro, inicio de sesión, sesión persistente y cierre de sesión (US-01 a US-04). Es el diseño previsto: los nombres exactos de los endpoints se fijarán al implementar cada historia.

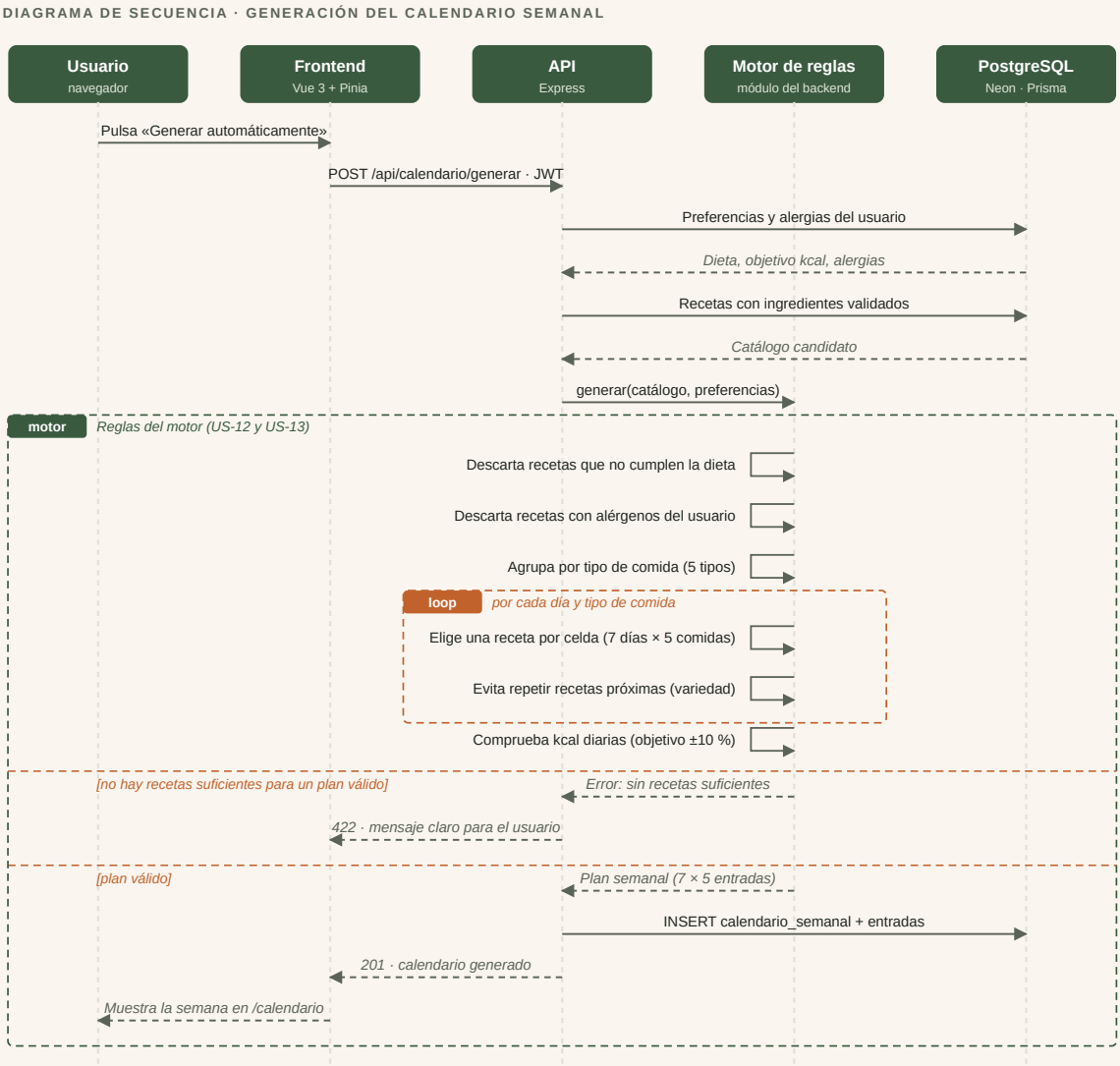
DIAGRAMA DE SECUENCIA · AUTENTICACIÓN (REGISTRO, LOGIN Y SESIÓN)



SECUENCIA · REGISTRO, LOGIN Y SESIÓN CON JWT

04 Secuencia: generación del calendario

> Cómo genera el motor de reglas el calendario semanal (US-12 y US-13): filtra por dieta y alergias, asigna una receta a cada celda evitando repeticiones y comprueba el objetivo calórico.



SECUENCIA · GENERACIÓN AUTOMÁTICA DEL CALENDARIO

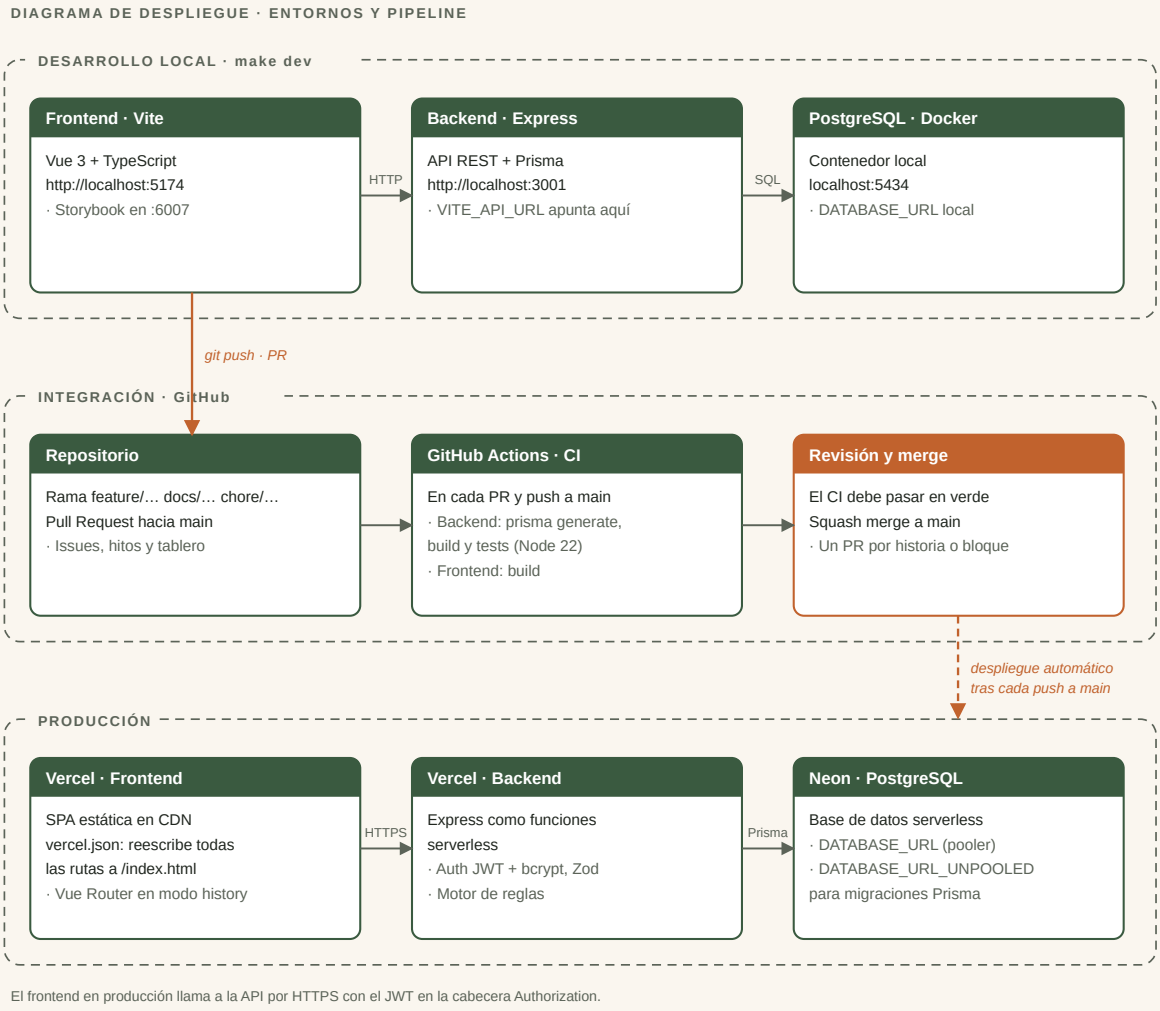
Entrada — preferencias, alergias y recetas con ingredientes validados.

Salida — 7 días × 5 comidas guardadas en entrada_calendario.

Fallo — si no hay recetas suficientes, se informa con un mensaje claro.

05 Diagrama de despliegue

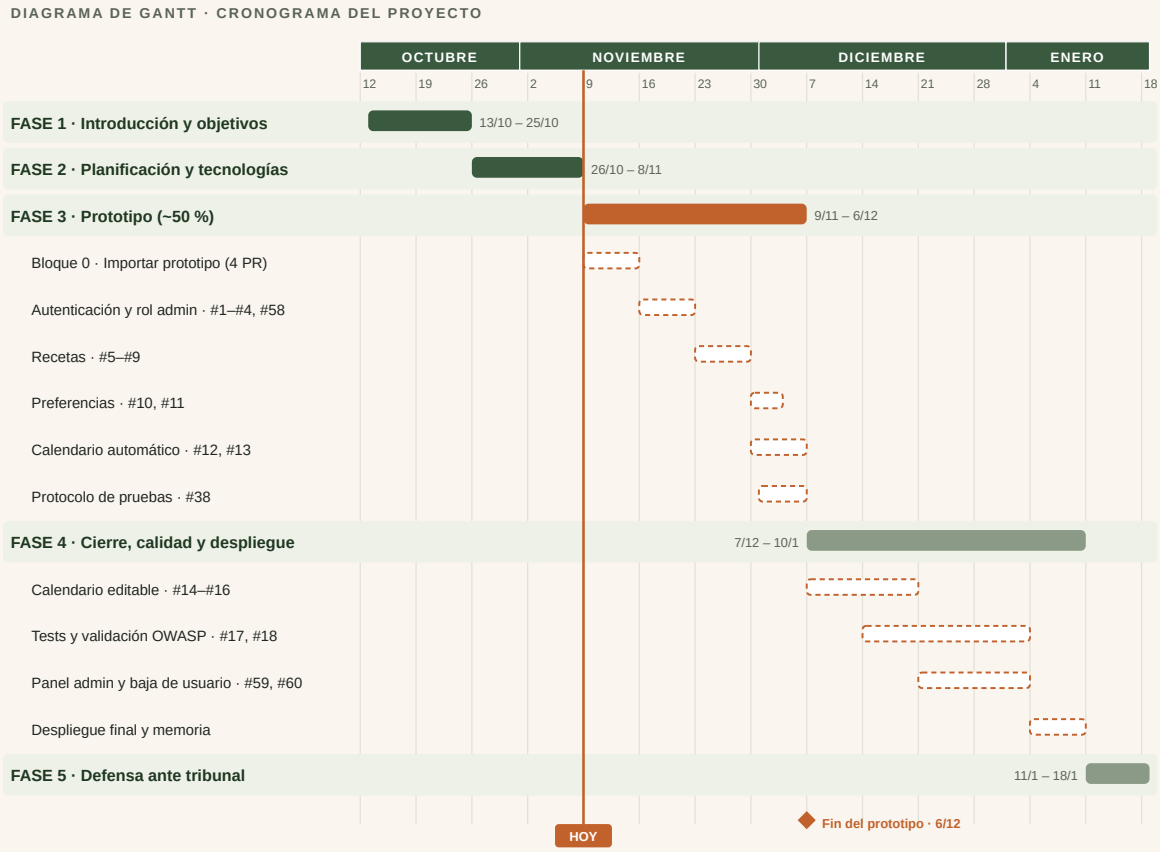
> Entornos del proyecto: desarrollo local con make dev, integración con GitHub y Actions, y producción en Vercel y Neon. Complementa el diagrama de arquitectura.



DESPLIEGUE · ENTORNOS Y PIPELINE

06 Diagrama de Gantt

> Cronograma de las cinco fases del proyecto con el desglose del prototipo y del cierre. Las fechas de las fases vienen de la planificación; el reparto semanal de las tareas es una previsión que se ajustará.



GANTT · OCTUBRE DE 2026 – ENERO DE 2027